Slovenská technická univerzita v Bratislave
Fakulta informatiky a informačných technológií

Predmet: Objektovo orientované programovanie

Bohdan Hahalyuk

Forest Knight

Checkpoint

Bratislava 2026

Obsah

Zámer projektu **3**
Herná mechanika **4**
Postavy **5**
Objekty v hre **6**
Funkcionálne požiadavky **7**
OOP princípy **8**
Generalizácia 8
Agregácia 9
Kompozícia 10
Dedenie 11
Polymorfizmus 12
Preťaženie 13
Prekonanie 14
UML diagram tried **15**
Popis 3-4 Unit testov **16**
testAttackDirectionRight 16
testAttackDirectionLeft 16
testAttackDealsDamage 17
testAttackCoolDown 17
Používateľský manuál **18**

Zámer projektu

- Hra sa odohráva v temnom lese po rozpade kráľovstva, kde sa nachádza mnoho nebezpečenstiev. Ide o survival arénu, v ktorej je cieľom zbierať body a prežiť do úsvitu.

- Hlavným hrdinom je rytier, ktorý utiekol žiť do lesov a musí tam prežiť. V lese sa pohybujú vlci a ďalší rytieri, ktorí sú však nepriateľskí. Tiež sa hovorí, že sa tam vyskytujú aj medvede.

- Hráč sa musí brániť pred nebezpečenstvami a šetriť šípy, aby bolo jednoduchšie prežiť v neskoršej fáze hry. Z každej porazenej bytosti je možné získať rôzny počet bodov a šípov.

- Pre víťazstvo je potrebné dosiahnuť určitý počet bodov v závislosti od zvolenej obtiažnosti.

- Hlavný hrdina má meč, ktorý sa nikdy nerozbije, a tiež kušu, ktorá bola v tom čase pomerne vzácna.

Herná mechanika

Pohyb hráča

- hráč sa pohybuje pomocou kláves WASD
- hra je z pohľadu zhora (top-down arena)

Útok

- hráč môže útočiť mečom (blízky boj)
- hráč môže používať kušu na streľbu šípov

Šípy

- počet šípov je obmedzený
- šípy je možné získať z nepriateľov

Nepriatelia

- nepriatelia sa pohybujú smerom k hráčovi
- pri dotyku spôsobujú poškodenie

Body

- za každého porazeného nepriateľa hráč získava body
- cieľom hry je dosiahnuť požadovaný počet bodov

Postavy

Hlavný hrdina – rytier (Player)

Hlavná postava hry. Pohybuje sa po mape, útočí mečom alebo kušou a snaží sa prežiť útoky nepriateľov.

Nepriatelia

• Vlci

Rýchli nepriatelia s menším množstvom života, útočia pri kontakte.

• Medvede

Pomalší, ale majú viac zdravia a spôsobujú väčšie poškodenie.

• Nepriateľskí rytieri

Majú strednú rýchlosť a môžu útočiť z diaľky.

Objekty v hre

V hre sa nachádzajú nasledujúce hlavné objekty (entity), ktoré budú reprezentované ako triedy v objektovo orientovanej implementácii.

Hlavné entity hry:

- Game – hlavná logika hry
- Player – hlavná postava hráča
- Enemy – abstraktná trieda nepriateľa
    - Wolf
    - Bear
    - KnightEnemy
- Arrow – projektil na diaľkový útok
- PowerUp – pomocné objekty
    - HealPotion
- WaveManager – riadenie vĺn nepriateľov

Funkcionálne požiadavky

High priority

- Pohyb hráča pomocou kláves WASD
- Útok mečom
- Streľba šípov
- Generovanie nepriateľov vo vlnách
- Kolízia medzi objektmi
- Systém bodovania
- Strata HP pri kontakte s nepriateľom

Medium priority

- Drop šípov z nepriateľov
- Healing power-up
- Rôzne typy nepriateľov
- Zvýšovanie obtiažnosti

Low priority

- Ďalšie power-upy
- Zvukové efekty
- Rozšírené herné mechaniky

OOP princípy

Generalizácia

Generalizácia predstavuje vytvorenie všeobecného (abstraktného) modelu pre viacero podobných objektov.
V projekte je tento princíp implementovaný pomocou abstraktnej triedy `Enemy`, ktorá obsahuje spoločné vlastnosti a metódy pre všetkých nepriateľov.

Trieda `Enemy` definuje základné atribúty ako napríklad `hp`, `speed`, `damage` a abstraktné metódy `update()` a `draw()`, ktoré musia byť implementované v konkrétnych triedach.

Z tejto triedy následne dedia konkrétne typy nepriateľov:

- `Wolf`
- `Bear`
- `KnightEnemy`

Týmto spôsobom sa zjednodušuje kód a zabraňuje sa jeho duplicite.

`Enemy:`

```
21 usages   3 inheritors   Bohdan Hahalyuk +1
public abstract class Enemy {
    protected int x = 0;
    protected int y = 0;
    protected int width = 0;
    protected int height = 0;

    15 usages
    protected int speed = 2;

    10 usages
    protected int hp = 0;

    4 usages
    protected int damage;

    4 usages
    protected boolean isAlive;
    4 usages
    protected int scoreValue;


    1 usage   3 implementations   Bohdan Hahalyuk
    public abstract void update(Player player);

    3 implementations   Bohdan Hahalyuk
    public abstract void draw(Graphics g);

    2 usages   Bohdan Hahalyuk
    public Rectangle getBounds() { return new Rectangle(x, y, width, height); }
    1 usage   Bohdan Hahalyuk
    public void takeDamage(Player player) { player.hp -= damage; }
}
```

Agregácia

Agregácia predstavuje vzťah medzi objektmi, kde jeden objekt obsahuje referencie na iné objekty, ale neovláda ich životný cyklus.

V projekte je agregácia použitá v triede `GamePanel`, ktorá obsahuje kolekciu nepriateľov:

```
ArrayList<Enemy> enemies
```

Trieda `GamePanel` pracuje s objektmi typu `Enemy`, napríklad ich aktualizuje, vykresľuje a kontroluje kolízie.
Tieto objekty však môžu existovať aj nezávisle od triedy `GamePanel`.

Podobne je agregácia použitá aj pri PowerUp objektoch:

```
ArrayList<PowerUp> powerUps
```

Týmto spôsobom je možné jednoducho spravovať viacero objektov rovnakého typu.

Vo `GamePanel:`

```
2 usages   Bohdan Hahalyuk +1 *
class GamePanel extends JPanel implements KeyListener, MouseListener {
    5 usages
    ArrayList<Enemy> enemies = new ArrayList<>();
    4 usages
    ArrayList<PowerUp> powerUps = new ArrayList<>();
```

```
2 usages   Bohdan Hahalyuk +1 *
public class WaveManager {
    6 usages
    ArrayList<Enemy> enemies;
    5 usages
    ArrayList<PowerUp> powerUps;
```

Vo `WaveManager:`

```
1 usage   Bohdan Hahalyuk +1 *
public WaveManager(ArrayList<Enemy> enemies, ArrayList<PowerUp> powerUps){
    this.enemies = enemies;
    this.powerUps = powerUps;
    this.waveNumber = 0;
}
```

Kompozícia

Kompozícia je silnejší vzťah medzi objektmi, kde jeden objekt vlastní druhý a riadi jeho životný cyklus.

V projekte je kompozícia implementovaná v triede `GamePanel`, ktorá obsahuje objekty:

- `Player player`
- `WaveManager waveManager`

Tieto objekty sú vytvárané a používané výhradne v rámci `GamePanel` a bez neho by nemali význam.

Ďalším príkladom kompozície je trieda `Player`, ktorá obsahuje zoznam aktívnych efektov:

```
ArrayList<TemporaryEffect> activeEffects
```

Tieto efekty existujú len počas života hráča a sú plne riadené triedou `Player`.

Kompozícia umožňuje lepšiu organizáciu kódu a jasné určenie zodpovedností medzi objektmi.

```
2 usages   Bohdan Hahalyuk +1 *
class GamePanel extends JPanel implements KeyListener, MouseListener {
    5 usages
    ArrayList<Enemy> enemies = new ArrayList<>();
    4 usages
    ArrayList<PowerUp> powerUps = new ArrayList<>();
    37 usages
    Player player = new Player();
    2 usages
    WaveManager waveManager = new WaveManager(enemies, powerUps);
```

```
Bohdan Hahalyuk +1 *
public class Player {
    4 usages
    ArrayList<TemporaryEffect> activeEffects = new ArrayList<>();
```

Dedenie

Dedenie umožňuje vytvárať nové triedy na základe existujúcich tried.
V projekte je dedenie využité pri vytváraní rôznych typov nepriateľov, ktoré dedia od abstraktnej triedy `Enemy`.

Každý typ nepriateľa (`Wolf`, `Bear`, `KnightEnemy`) preberá spoločné vlastnosti z triedy `Enemy`, ale zároveň si môže definovať vlastné správanie.

Okrem toho je dedenie použité aj pri PowerUp systéme:

- abstraktná trieda `PowerUp`
- konkrétne triedy ako `DamageIncreasePower`, `HealthPower`, `SpeedIncreasePower, SwordPower`

Týmto sa zabezpečuje prehľadná hierarchia tried a jednoduchá rozšíriteľnosť projektu.

```
public abstract class Enemy {
    public class Wolf extends Enemy{
    public class Bear extends Enemy{
    public class KnightEnemy extends Enemy{
```

```
public abstract class PowerUp {
    public class DamageIncreasePower extends PowerUp {
    public class HealthPower extends PowerUp{
    public class SpeedIncreasePower extends PowerUp{
    public class SwordPower extends PowerUp{
```

Polymorfizmus

Polymorfizmus umožňuje pracovať s rôznymi objektmi prostredníctvom spoločného rozhrania.

V projekte je polymorfizmus využitý viacerými spôsobmi.

Run-time polymorfizmus je implementovaný pri práci s nepriateľmi. Trieda `GamePanel` pracuje s kolekciou typu `ArrayList<Enemy>`, pričom každý objekt môže byť konkrétny typ ako `Wolf`, `Bear` alebo `KnightEnemy`. Pri volaní metódy `update()` alebo `draw()` sa vykoná implementácia konkrétnej triedy.

Podobný princíp je použitý aj pri PowerUp objektoch, kde sa cez typ `PowerUp` volá metóda `apply(Player player)`, pričom správanie závisí od konkrétneho typu PowerUp.

Týmto spôsobom je možné rozširovať projekt bez nutnosti meniť existujúci kód.

Vo `PowerUp`:

```
1 usage   4 implementations   new *
public abstract void apply(Player player);
```

Vo napr. `DamageIncreasePower`:

```
1 usage   new *
@Override
public void apply(Player player) { player.damage += extraDamage; }
```

Vo napr. `HealthPower`:

```
1 usage   new *
@Override
public void apply(Player player) {player.hp += extraHp;}
```

Preťaženie

Preťaženie metód znamená, že viacero metód má rovnaký názov, ale líšia sa v parametroch.

V projekte je preťaženie implementované v triede `Player` pri metóde `attack()`.

Existujú dve verzie metódy:

- `attack(ArrayList<Enemy> enemies, int mouseX, int mouseY)`
- `attack(ArrayList<Enemy> enemies, int mouseX, int mouseY, int extraDamage)`

Prvá verzia vykonáva základný útok, zatiaľ čo druhá verzia umožňuje použiť dodatočné poškodenie, napríklad pri aktivovaní PowerUp.

Tento prístup zvyšuje flexibilitu kódu a umožňuje jednoduchšie rozširovanie funkcionality.

PowerUp nemení priamo útok, ale mení stav hráča (bonusDamage), ktorý sa následne využíva v preťaženej metóde attack():

```
4 usages   Bohdan Hahalyuk *
public void attack(ArrayList<Enemy> enemies, int mouseX, int mouseY){
    attack(enemies, mouseX, mouseY, extraDamage);
}

2 usages   new *
public void attack(ArrayList<Enemy> enemies, int mouseX, int mouseY, int extraDamage) {
    if (attackCoolDown == 0) {
        Rectangle attackRect = getAttackBounds(mouseX, mouseY, attackSize);
        for (Enemy enemy : enemies) {
            if (collisionEnable(enemy, attackRect)) {
                enemy.hp -= damage + extraDamage;
                attackCoolDown = 30;
            }
        }
    }
    if (attackCoolDown > 0) attackCoolDown--;
}
```

Prekonanie

Prekonanie znamená, že podtrieda poskytne vlastnú implementáciu metódy, ktorá je definovaná v nadtriede.

V projekte je overriding použitý v triedach, ktoré dedia od `Enemy`. Každý nepriateľ implementuje vlastnú verziu metódy `update()` a `draw()`.

Napríklad:

- `Wolf` sa pohybuje rýchlejšie
- `Bear` má viac života a je pomalší
- `KnightEnemy` má odlišné správanie

Rovnako je overriding použitý aj pri triedach `PowerUp`, kde každá trieda implementuje vlastnú metódu `apply(Player player)`.

Týmto sa zabezpečuje rôzne správanie objektov pri zachovaní jednotného rozhrania.

```
19 usages   3 inheritors   Bohdan Hahalyuk +1 *
public abstract class Enemy {
    protected int x = 0;
    protected int y = 0;
    protected int width = 0;
    protected int height = 0;
    15 usages
    protected int speed = 2;
    9 usages
    protected int hp = 0;
    4 usages
    protected int damage;
    4 usages
    protected boolean isAlive;
    4 usages
    protected int scoreValue;

    1 usage   3 implementations   Bohdan Hahalyuk
    public abstract void update(Player player);

    3 implementations   Bohdan Hahalyuk
    public abstract void draw(Graphics g);
```

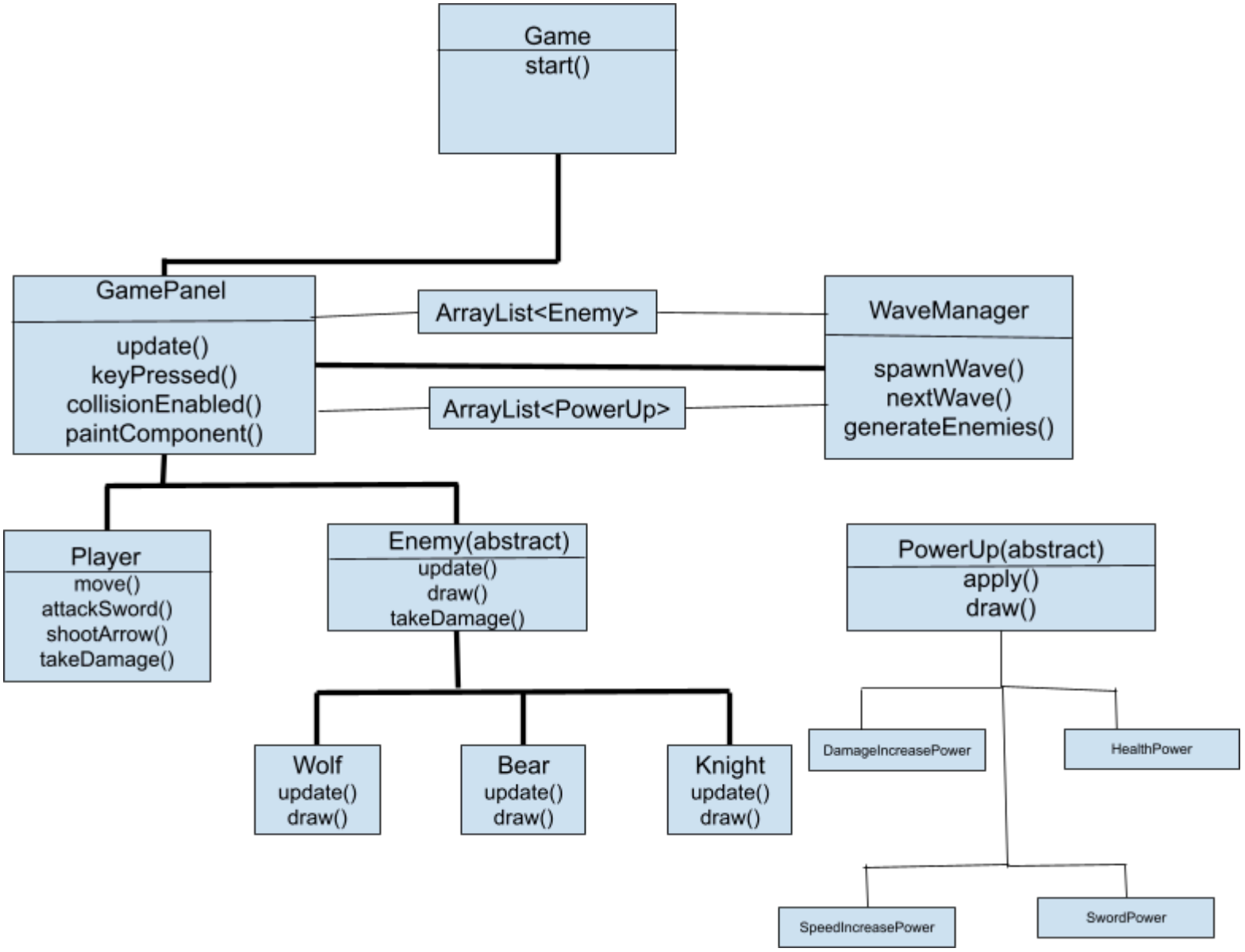
```
3 usages   Bohdan Hahalyuk *
public class Wolf extends Enemy{

    1 usage   Bohdan Hahalyuk *
    @Override
    public void update(Player player) {
        if(player.x > x){x += speed;}
        if(player.y > y){y += speed;}
        if(player.x < x){x -= speed;}
        if(player.y < y){y -= speed;}
    }

    Bohdan Hahalyuk
    @Override
    public void draw(Graphics g) { g.drawRect(x, y, width, height); }

    3 usages   Bohdan Hahalyuk
    public Wolf(int x, int y) {
        this.x = x;
        this.y = y;
        width = 20;
        height = 20;
        speed = 3;
        damage = 2;
        scoreValue = 30;
        hp = 20;
        isAlive = true;
    }
}
```

UML diagram tried



Popis 3-4 Unit testov

testAttackDirectionRight

- Účel: Skontrolovať, či metóda getAttackDirection() správne určí smer útoku, keď sa kurzor nachádza napravo od hráča.
- Vstup: hráč na (50, 50), kurzor na (100, 50)
- Očakávaný výsledok: Smer DOPRAVA

```
new *
@Test
void testAttackDirectionRight() {
    Player player = new Player();
    player.x = 50;
    player.y = 50;

    Player.AttackDirection dir = player.getAttackDirection( mouseX: 100,  mouseY: 50);

    assertEquals(Player.AttackDirection.RIGHT, dir);
}
```

testAttackDirectionLeft

- Cieľ: skontrolovať smer útoku doľava
- Vstup: hráč na (50, 50), kurzor na (0, 50)
- Očakávaný výsledok: Smer DOĽAVA

```
new *
@Test
void testAttackDirectionLeft() {
    Player player = new Player();
    player.x = 50;
    player.y = 50;

    Player.AttackDirection dir = player.getAttackDirection( mouseX: 0,  mouseY: 50);

    assertEquals(Player.AttackDirection.LEFT, dir);
}
```

testAttackDealsDamage

- Cieľ: skontrolovať, či útok hráča znižuje HP nepriateľa
- Vstup: hráč na (50, 50), nepriateľ (60, 50)
- Očakávaný výsledok: HP nepriateľa je menšie ako pred útokom

```
new *
@Test
void testAttackDealsDamage() {
    Player player = new Player();
    player.x = 50;
    player.y = 50;

    Enemy enemy = new Wolf( x: 60,  y: 50);
    int initialHp = enemy.hp;

    ArrayList<Enemy> enemies = new ArrayList<>();
    enemies.add(enemy);

    player.attack(enemies,  mouseX: 100,  mouseY: 50);

    assertTrue( condition: enemy.hp < initialHp);
}
```

testAttackCoolDown

- Cieľ: skontrolovať, či mechanika cooldownu funguje - opätovný útok pred skončením cooldownu nepriateľa nepoškodí
- Vstup: rovnaká pozícia hráča a nepriateľa
- Očakávaný výsledok: HP po druhom útoku sa nemení

```
new *
@Test
void testAttackCoolDown() {
    Player player = new Player();
    player.x = 50;
    player.y = 50;

    Enemy enemy = new Wolf( x: 60,  y: 50);

    ArrayList<Enemy> enemies = new ArrayList<>();
    enemies.add(enemy);

    player.attack(enemies,  mouseX: 100,  mouseY: 50);
    int hpAfterFirstHit = enemy.hp;

    player.attack(enemies,  mouseX: 100,  mouseY: 50);

    assertEquals(hpAfterFirstHit, enemy.hp);
}
```

Používateľský manuál

Spustenie programu:
Program sa spúšťa cez triedu `Main`, ktorá inicializuje hru pomocou metódy `Game.start()`.

Ovládanie:

- Pohyb hráča: klávesy W, A, S, D
- Útok: ľavé tlačidlo myši (LKM)
- Ovládanie meča: V závislosti od smeru myši

Cieľ hry:
Cieľom hry je prežiť čo najdlhšie a získať čo najviac bodov porážaním nepriateľov.

Herné prvky:

- Nepriatelia sa objavujú vo vlnách
- PowerUp objekty zlepšujú schopnosti hráča (napr. damage, speed, health)